VirPet Project

Lecture Note — Game model, event cards, Swing UI, and Android port
Stat management, file-based card loading, sprite composition, listeners, and screen navigation

Classes covered: `PetGameModel`, `StatDelta`, `EventCard`, `EventCardLoader`, `EventCardDeck`, `PetCreationApp`, `PetSpriteComposer`, `GameAudio`, `androidPort/.../VirPetActivity.java`

Goal: Explain each class and important methods in an easy-to-follow way, using the same lecture-note format as the instructor's Tetris example (Class/Method — Purpose — Teaching point).

Student	Emine Mihrinaz Kurt
Student ID	24040102017
GitHub repository	https://github.com/CortXVenomSkulls/virPet
Related documents	`README.md`, `manual.html`, `rapor.html` (Turkish)

Table of contents

1. Big Picture: How the Project Is Organized

This version is **component-based**, similar to the instructor's Tetris split into `game`, `ui`, and `model` packages. VirPet uses a single default package; responsibilities are separated by class. The application class controls navigation; the model holds stat rules; the card system reads external data; UI classes build Swing screens and drawing.

Class roles

Class / file	Purpose	Teaching point
`PetCreationApp`	Application entry point and screen controller.	Connects creation screen, game screen, Yes/No, stat bars, audio, and restart.
`PetGameModel`	Pure numeric game rules.	Stats, time passage, loss conditions, PNG paths from weight/affection; no Swing.
`EventCardLoader` / `EventCardDeck`	Load external cards and draw randomly.	Content outside code; edit `.cards` only to add events.
`PetSpriteComposer`	Merge part PNGs into one sprite.	16×48 canvas, layer order, drawing with `Graphics2D`.
`GameAudio`	Play WAV via JDK `Clip`.	Looping background and short SFX; no external player.
`androidPort/.../VirPetActivity`	Mobile edition of the same game.	Activity + View; `AssetManager` and `MediaPlayer` (MP3).

Important design idea

> **Model–UI separation:** `PetGameModel` does not know card text or buttons; it only receives a `StatDelta` and updates stats. The UI reads the model and redraws. This mirrors the Tetris `GameListener` bridge between `TetrisGame` and `GameView` — in VirPet the bridge is direct method calls in `PetCreationApp`.

Simplified runtime flow

```
App starts → Creation (parts + name) → Start → Event card
    → Yes/No → PetGameModel.applyChoice → Time passage → Lost?
    → No: next card | Yes: Game over → Play again → Creation
```

Project tree

```
virPet/
├── PetCreationApp.java
├── PetGameModel.java
├── PetSpriteComposer.java
├── StatDelta.java
├── EventCard.java
├── EventCardLoader.java
├── EventCardDeck.java
├── GameAudio.java
├── assets/          (parts/, event_cards.cards, audio, font, bg.png)
├── androidPort/
└── manual.html, rapor.html, rapor-en.html
```

2. Game Model Classes

This layer acts as the game engine: no login forms or file persistence; stat rules and sprite path resolution live here.

2.1 StatDelta.java

Carries the effect of one event card choice on four stats. Omitted fields in the card file count as 0.

Element	Purpose	Teaching point
`hunger, affection, weight, health`	Four public final fields.	Immutable data object used when applying a choice.
`StatDelta(...)`	Constructor.	`this.field = parameter` assignment pattern.
`StatDelta.ZERO`	All deltas zero.	Shared constant; loader uses it for missing yes/no lines.

```
public static final StatDelta ZERO = new StatDelta(0, 0, 0, 0);
```

2.2 EventCard.java

One Reigns-style dilemma: id, text, and two `StatDelta` values for yes and no.

Method	Purpose	Teaching point
`EventCard(id, text, yes, no)`	Builds one card.	`private final` fields — encapsulation.
`getId()`, `getText()`, `getYes()`, `getNo()`	Read-only access.	Loaded file data cannot be changed from outside.

2.3 PetGameModel.java

Core logic: stat changes, passive time, game-over messages, body and face PNG paths from weight and affection.

Constants (thresholds)

Constant	Value	Teaching point
`STAT_MIN` / `STAT_MAX`	0 / 100	`clamp()` prevents overflow.
`WEIGHT_SKINNY` / `WEIGHT_FAT`	35 / 65	Which body image (skinny / normal / fat).
`AFFECTION_SAD` / `AFFECTION_HAPPY`	35 / 65	Which face image (sad / normal / happy).
`TICK_HUNGER` etc.	+3, −2, −1, −2	Automatic per-turn effect after each choice.

Method summary

Method	Purpose	Teaching point
`PetGameModel(...)`	Part paths, style ids, name; initial stats.	Name `trim`; default `"Pet"` if empty — ternary operator.
`applyChoice(StatDelta)`	Applies choice, then `applyTimePassage()`.	Early return if game over or null delta.
`resolveBodyPath()` / `resolveFacePath()`	`Path` from current weight/affection.	Chain: `partsRoot.resolve("bodies").resolve(idStr)`.
`getGameOverMessage()`	User-facing English message.	`switch` on `GameOverReason` enum.
`evaluateLossConditions()` (private)	Affection ≤0 or health ≤0.	`LEFT_HOME`, `OBESITY`, `STARVATION`.

```
public enum GameOverReason {
    NONE, OBESITY, STARVATION, LEFT_HOME
}
```

Why use an enum? Typos are caught at compile time; safer than string constants (same emphasis as in the Tetris lecture notes).

3. Event Card Loading and Deck

3.1 EventCardLoader.java

Reads `assets/event_cards.cards` line by line and builds a `List<EventCard>`.

Method	Purpose	Teaching point
`private EventCardLoader()`	Blocks instantiation.	Utility class: only `load` is used.
`load(Path path)`	Parses the file.	`throws IOException`; try-with-resources closes `BufferedReader`.
`parseDelta(String)` (private)	Parses lines like `hunger=-14 affection=3`.	Empty rest → `StatDelta.ZERO`.

File format (example)

```
card empty_bowl
text
The food bowl is empty. Your pet looks at you with big eyes. Fill it up?
yes hunger=-14 affection=3 weight=6
no hunger=6 affection=-6
end
```

44 cards total. Lines starting with `#` and blank lines are ignored.

3.2 EventCardDeck.java

Method	Purpose	Teaching point
`EventCardDeck(List)`	Rejects empty list.	`Collections.unmodifiableList` protects internal list.
`drawNext()`	Returns random card.	`do-while`: avoids same card twice in a row when possible.

4. UI Layer

Swing builds screens; user actions are forwarded to the model and deck.

4.1 PetCreationApp.java

Main class: `CardLayout` for *creation* and *game* panels; inner classes `CreationPreviewPanel`, `GamePreviewPanel`, `StatBarsPanel`.

Method / structure	Purpose	Teaching point
`main(String[])`	Program entry.	Font load, card list, `EventCardDeck`, show frame.
`buildCreationCard()` / `buildGameCard()`	Build both screens.	Arrow labels cycle parts; stat bars and event text on game card.
`startGame()`	Creates `PetGameModel`, starts music.	Model built from name field; first card drawn.
`resolveEvent(boolean yes)`	Handles Yes/No.	`model.applyChoice` → `afterGameStep()` → refresh bars/sprite.
`showGameOverInEventPanel()`	Loss message and sound.	`death.wav` or `runaway.wav`.
`returnToCreation()`	Play again.	Reset model; switch back to creation card.
`installTapSounds(JFrame)`	Global click sound.	`AWTEventListener` on all mouse clicks.

4.2 PetSpriteComposer.java

Method	Purpose	Teaching point
`SPRITE_W` / `SPRITE_H`	16×48 canvas.	`BufferedImage.TYPE_INT_ARGB` for transparency.
`compose(hat, leg, body, face)`	Stacks four PNGs.	Legs y=32, body/face y=16, hat y=0.
`drawAt(g, path, x, y)` (private)	Draws one part.	`ImageIO.read`; missing files skipped quietly.

4.3 GameAudio.java

Method	Purpose	Teaching point
`startBgMusic(Path)`	Loop `bg.wav`.	`Clip.LOOP_CONTINUOUSLY`.
`playTap` / `playDeath` / `playRunaway`	Short SFX.	Static `sfxClips` list; listener closes clip when done.
`shutdown()`	Cleanup on exit.	Called from `windowClosing`.

5. Assets

Layer	Y	Selection
Legs	32	`legsN.png`
Body	16	Weight → skinny / normal / fat
Face	16	Affection → sad / normal / happy
Hat	0	`hatN.png`

Audio: WAV on desktop; same files as MP3 under `app/src/main/assets/` on Android.

6. Android Port

`VirPetActivity.java` packs creation, game, and embedded model/card/audio logic in one file for portability; rules match the desktop build.

Feature	Desktop	Android
UI	Swing	LinearLayout, custom `View`
Audio	`javax.sound.sampled`	`MediaPlayer`
Files	`java.nio.file`	`AssetManager`

7. Running the Project

```
cd virPet
javac -source 8 -target 8 *.java
java PetCreationApp
```

```
cd androidPort
export ANDROID_HOME=~/Android/Sdk
./gradlew assembleDebug
```

Repository: https://github.com/CortXVenomSkulls/virPet

8. Suggested Teaching Sequence

Step	Topic	Teaching point
1	`StatDelta` and `EventCard`	Data objects and encapsulation.
2	`EventCardLoader` and `.cards` format	File I/O, try-with-resources.
3	`PetGameModel` constants and `applyChoice`	Stats and time passage.
4	`resolveBodyPath` / sprite thresholds	Conditional file paths.
5	`PetSpriteComposer`	Canvas and layered drawing.
6	`PetCreationApp` creation screen	Swing layout, preview panel.
7	Game screen and `resolveEvent`	Model–UI connection.
8	`GameAudio` and Android port	Platform differences.

9. Final Summary

VirPet combines object-oriented design, Swing UI, file-based data, enums, the `Path` API, custom panel rendering, a turn-based game loop, and an optional Android build in one compact application — documented class-by-class with tables in the same style as the instructor’s Tetris lecture notes.

Student: Emine Mihrinaz Kurt (24040102017) · **GitHub:** CortXVenomSkulls/virPet